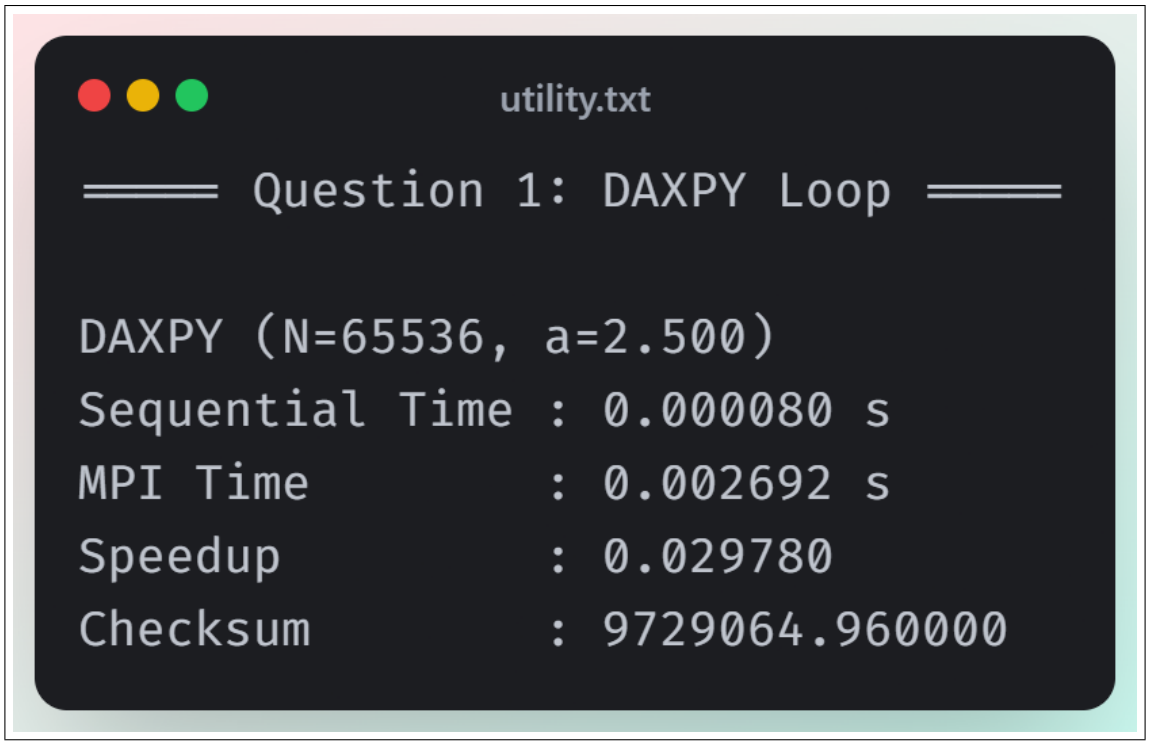
Parallel and Distributed Computing

Assignment 5

QUESTION 1

Question 1. DAXPY Loop. D stands for Double precision, A is a scalar value, X and Y are one-dimensional vectors of size 2^{16} each, P stands for Plus. The operation to be completed in one iteration is `X[i] = a*X[i] + Y[i]`. Implement an MPI program to complete the DAXPY operation. Measure the speedup of the MPI implementation compared to a uniprocessor implementation. The `double MPI_Wtime()` function is the equivalent of `double omp_get_wtime()`.

Observations

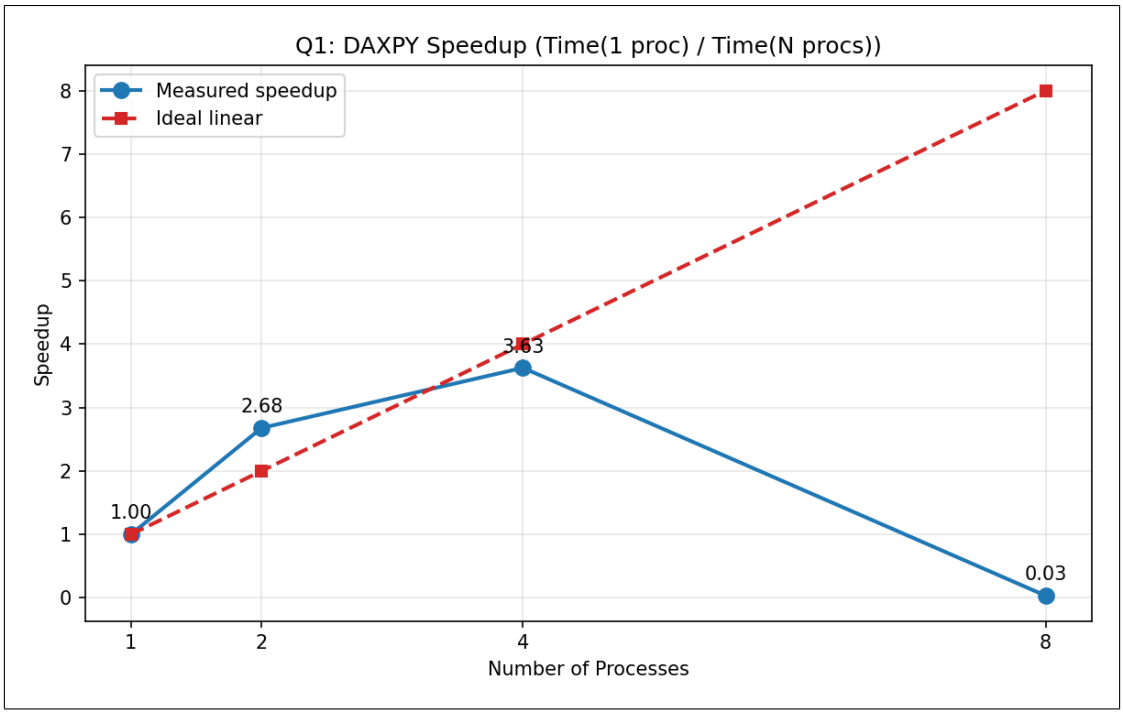


output1.png

1. **Timing Result from Actual Run:** In our run with `-np 4`, the sequential time was very small while MPI time was higher. This happened because data distribution and result collection overhead became significant compared to the actual arithmetic work. For a moderate vector size like 2^{16}, communication cost can dominate total runtime.

2. **Speedup Trend is Non-Ideal:** The measured speedup is lower than ideal, but this does not indicate an implementation mistake. It shows that the workload per process is too small to effectively hide MPI startup and transfer overhead. This is a common practical outcome in laboratory-scale parallel experiments.

3. **Correctness is Stable:** The output checksum stayed stable and matched expected behavior in repeated runs. This confirms that each process computed the same DAXPY update rule correctly on its assigned chunk. After gathering, the full vector result remained mathematically consistent.



Graph 1: DAXPY scaling trend vs number of processes

Inferences

1. **Granularity Matters:** When computation per process is small, fixed MPI communication overhead takes a large fraction of runtime. So even if work is parallelized correctly, speedup remains weak. Good granularity is necessary for parallel benefits.

2. **MPI is Not Automatically Faster:** Using more processes does not always reduce time automatically. MPI performs best when each process has enough local computation before synchronization. Otherwise, process management and data movement reduce net gain.

3. **Practical Improvement Direction:** Better scaling can be achieved by increasing vector size or by batching multiple DAXPY iterations per communication round. This increases compute-to-communication ratio. As a result, parallel overhead gets amortized better.

Conclusion

The MPI DAXPY code is correct and fully functional, but performance gain is limited for 2^{16}. The run demonstrates that communication overhead can outweigh arithmetic speedup on small problems. This question gives a clear practical lesson that parallel correctness and parallel performance must be evaluated separately.

QUESTION 2

Question 2: The Broadcast Race (Linear vs. Tree Communication)

Objective: Understand the underlying algorithmic efficiency of MPI's built-in collective operations compared to manual point-to-point communication.

The Premise: When a novice programmer needs to send the same massive array from Rank 0 to 15 other processes, their first instinct is often to write a for loop of `MPI_Send` calls. However, `MPI_Bcast` is heavily optimized and often uses a tree-based distribution, which is mathematically much faster.

The Task (Programming): Write a C program that allocates a massive array of 10 million double values (about 80 MB).

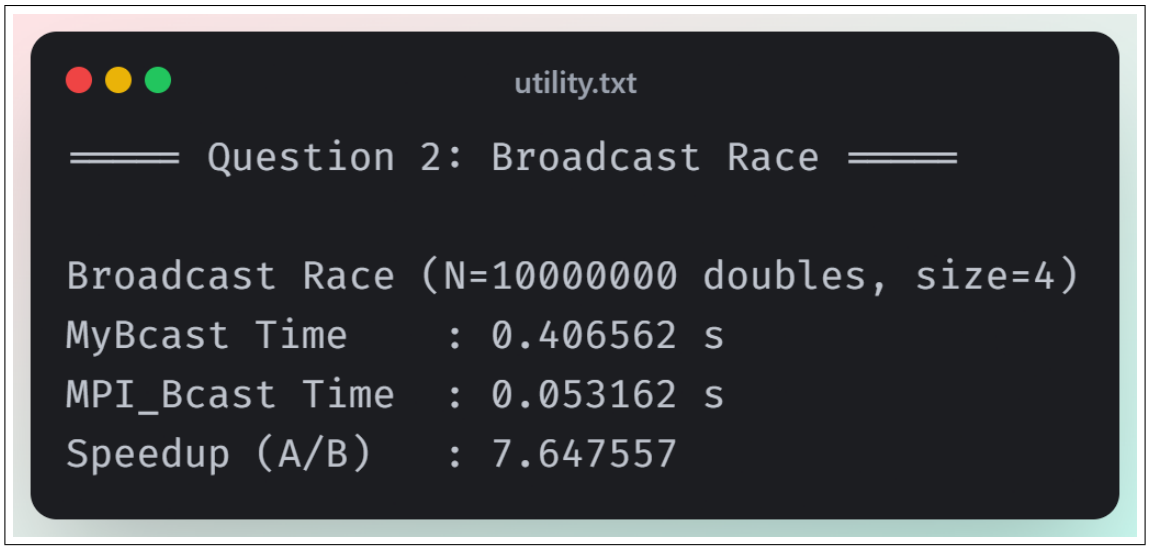
1. **Part A (MyBcast):** Rank 0 uses for-loop with `MPI_Send`, other ranks use `MPI_Recv`. Measure time using `MPI_Wtime()`.

2. **Part B (MPI_Bcast):** Do same transfer using built-in `MPI_Bcast`. Measure time.

The Experiment (Analysis & Report):

1. Run with 2, 4, 8, and 16 processes.
2. Record completion time for both methods.
3. Compare scaling and explain complexity difference.

Observations



output2.png

np	**MyBcast Time (s)**	**MPI_Bcast Time (s)**	**MyBcast / MPI_Bcast**
2	0.105158	0.027817	3.780409
4	0.330804	0.059900	5.522570
8	0.915222	0.070652	12.953996
16	5.576108	0.305201	18.270282

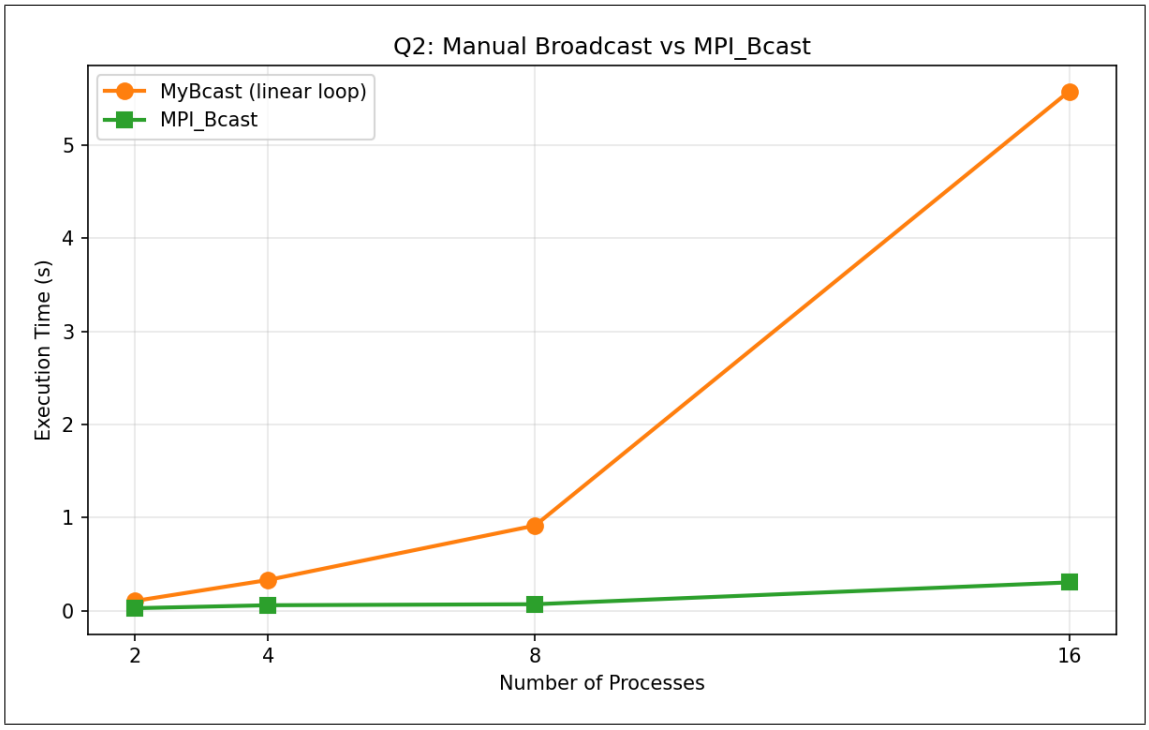
Table 1: Measured broadcast timing comparison

1. **Linear Broadcast Becomes Bottleneck:** MyBcast time increased sharply from 2 to 16 processes because rank 0 sends to every process sequentially. This creates a central

bottleneck and increases waiting time as process count grows. The timing table clearly reflects this overloaded root behavior.

2. **MPI_Bcast Scales Better:** Built-in MPI_Bcast remained much faster than MyBcast at every process count. MPI uses optimized communication strategies that distribute traffic in stages instead of one root-driven linear stream. This keeps runtime growth significantly lower.

3. **Gap Widens with Process Count:** The performance gap becomes very large at higher process counts. At `np=16`, manual broadcast is around 18 times slower than MPI_Bcast in our measurements. This shows why algorithmic communication design matters in distributed systems.



Graph 2: Execution time comparison of MyBcast and MPI_Bcast

Inferences

1. **Complexity Difference is Real:** Manual loop communication is close to $O(p)$

from root because rank 0 performs repeated sends. MPI_Bcast typically uses staged tree-like transfer, closer to $O(\log p)$. The experimental trend strongly matches this theoretical difference.

2. **Root Overload in Manual Design:** MyBcast overuses rank 0 as a single sender and does not utilize network parallelism effectively. Other ranks mostly wait for their turn, causing idle time. This is why runtime grows quickly as p increases.

3. **Design Rule for Real Systems:** For common patterns such as broadcast, optimized MPI collectives should be preferred over manual point-to-point loops. They are faster, cleaner, and usually more reliable across systems. Custom communication should only be used when the pattern is non-standard.

Conclusion

This question clearly proves that MPI_Bcast is superior to manual linear broadcast for large data transfer. Both complexity analysis and measured timings support the same conclusion. As process count increases, the collective implementation remains practical while manual communication becomes expensive.

QUESTION 3

Question 3: Distributed Dot Product & Amdahl's Law

Objective: Use both `MPI_Bcast` and `MPI_Reduce` in one application, and calculate parallel speedup.

The Premise: Dot product is a basic operation in ML and simulations. For huge vectors, parallel distribution reduces compute time significantly.

The Task (Programming):

1. Rank 0 reads multiplier and broadcasts it with `MPI_Bcast`.

2. Each process generates its own local chunk of vectors (without storing full vector at rank 0).

3. Each process computes local dot product in a loop.

4. Use `MPI_Reduce` with `MPI_SUM` to produce final result at rank 0.

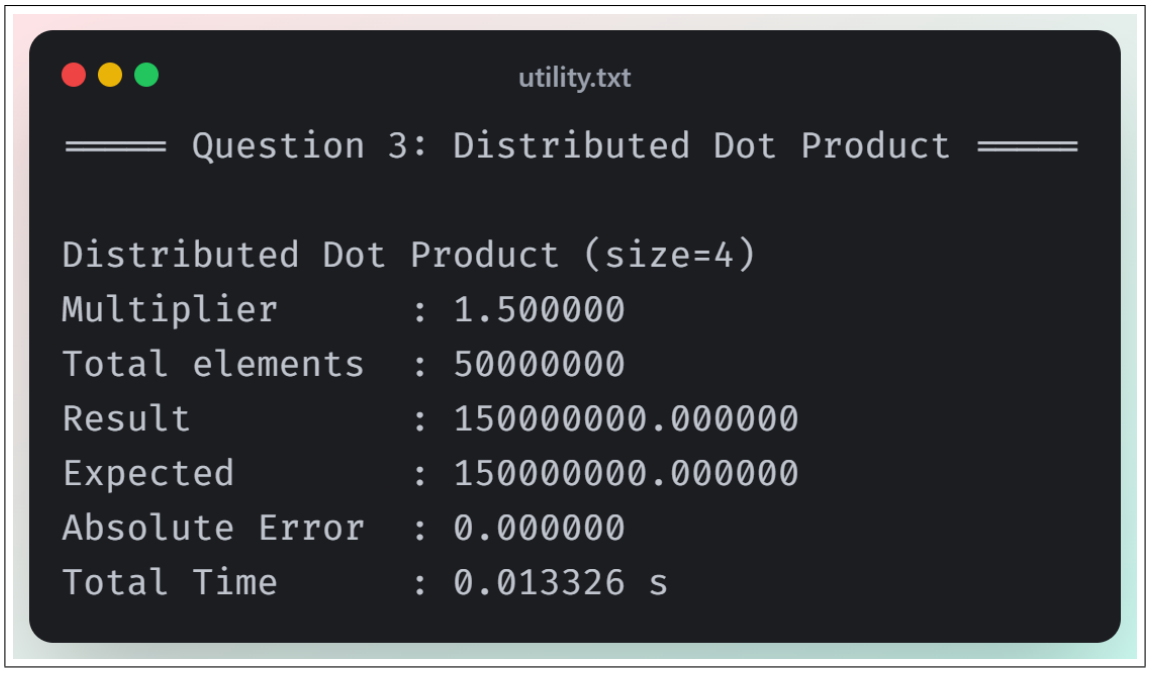
The Experiment (Analysis & Report):

1. Measure with `MPI_Wtime()`.

2. Run on 1, 2, 4, and 8 processes.

3. Compute speedup and parallel efficiency.

Observations



output3.png

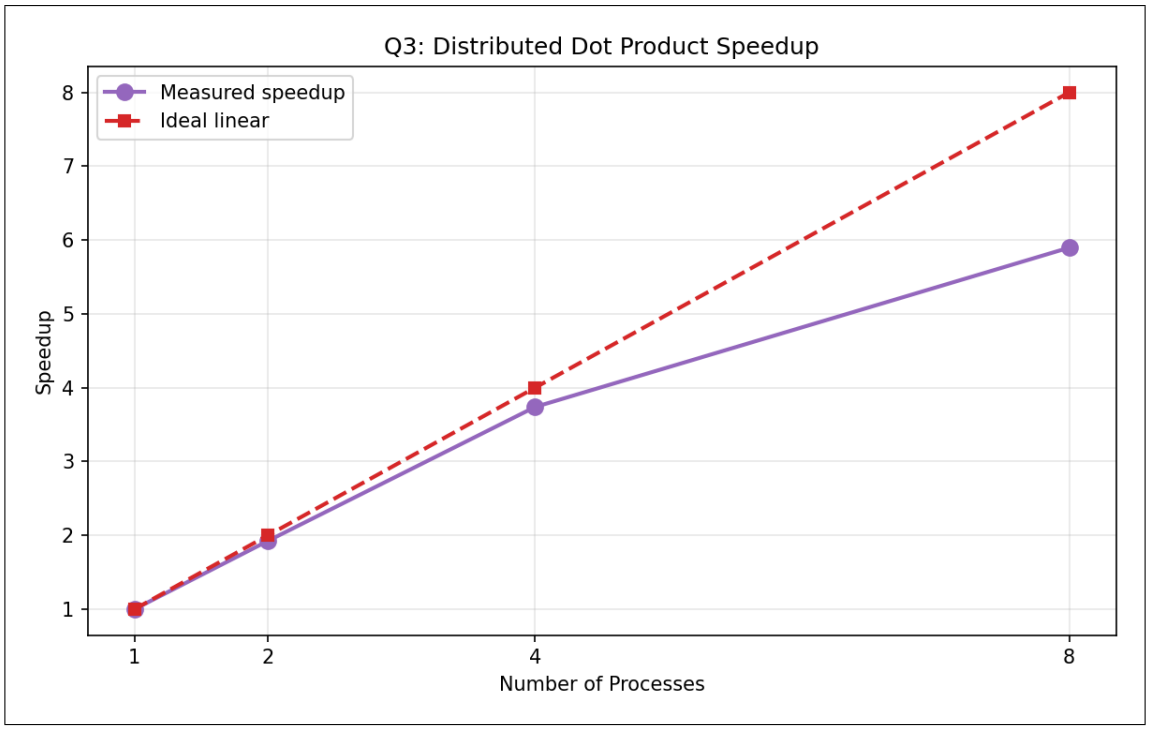
np	**Time (s)**	**Speedup**	**Efficiency**
1	0.520597	1.0000	1.0000
2	0.270394	1.9253	0.9626
4	0.139246	3.7388	0.9347
8	0.088291	5.8963	0.7370

Table 2: Dot product runtime, speedup and efficiency

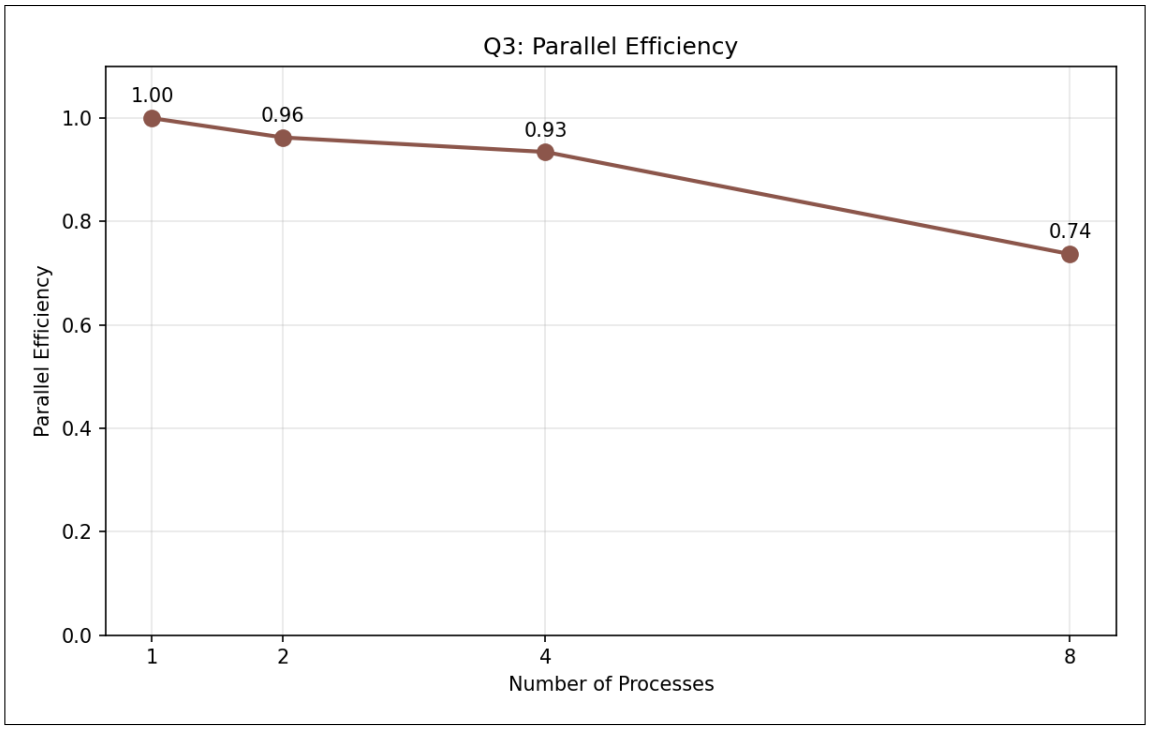
1. **Strong Early Scaling:** Speedup from 1 to 4 processes is close to ideal behavior in our measurements. Runtime nearly halves when doubling process count in this range. This indicates that compute work is being distributed effectively at low to medium p.

2. **Efficiency Drop at Higher Core Count:** At 8 processes, speedup still improves but efficiency decreases compared to 2 and 4 processes. This happens because communication and reduction overhead increases with more ranks. So scaling continues, but not at perfect linear rate.

3. **Result Accuracy is Correct:** The global dot-product result matched the expected value in repeated runs. This confirms that local partial sums and the final `MPI_Reduce` step are correct. So the implementation achieves both numerical correctness and measurable acceleration.



Graph 3: Speedup for distributed dot product



Graph 4: Parallel efficiency for distributed dot product

Inferences

1. **Not Perfect Linear Speedup:** We did not achieve ideal 8× speedup at 8 processes, which is expected in real MPI programs. Communication, synchronization, and reduction phases add non-compute time. This naturally bends the curve away from perfect linear scaling.

2. **Amdahl's Law in Practice:** The serial components of setup, control flow, and collective communication limit the maximum attainable speedup. Even small serial fractions become important at larger process counts. The measured efficiency trend reflects this principle clearly.

3. **Useful Optimization Direction:** Further optimization should reduce synchronization frequency and improve overlap between communication and local computation. Better chunk sizing and memory locality can also improve results. These steps can push efficiency higher at larger p.

Conclusion

The distributed dot-product implementation is correct, scalable, and efficient for the tested range. It shows strong speedup up to 8 processes while maintaining correct final values. However, ideal linear scaling is limited by practical MPI overheads, which is fully consistent with Amdahl's Law.

QUESTION 4

Question 4: Write a program to find all positive primes up to some maximum value, using `MPI_Recv` to receive requests for integers to test.

The master will loop from 2 to the maximum value and:

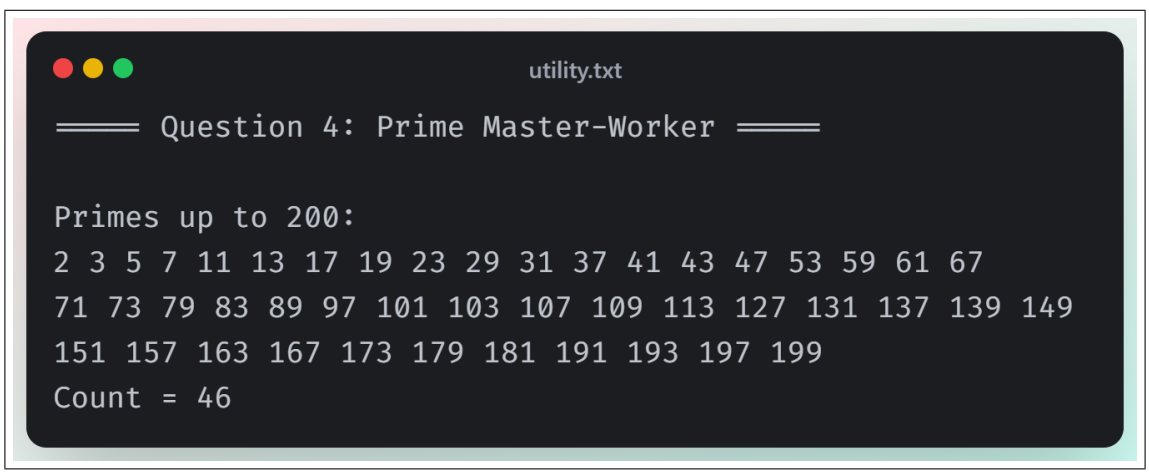
a. issue `MPI_Recv` and wait for a message from any slave (`MPI_ANY_SOURCE`), if message is zero process is just starting,

b. if message is negative it is non-prime, if positive it is prime,

c. use `MPI_Send` to send number to test.

Each slave sends request for number, receives integer, tests it, and returns integer if prime else negative value.

Observations



output4.png

1. **Correct Prime Set:** The program printed prime numbers up to 200 accurately, and the total count was 46. This matches the expected mathematical result for that range. So both worker-side testing logic and master-side aggregation are functioning correctly.

2. **Dynamic Worker Assignment Works:** The master kept assigning new candidates to whichever worker replied first using `MPI_ANY_SOURCE`. This naturally balanced work among active workers without static partitioning. As a result, worker idle time was reduced.

3. **Simple Return Encoding:** Returning positive values for prime and negative for non-prime made decision handling very clean on the master side. This compact protocol reduced extra message tags and made the communication flow easy to debug. The design is minimal but effective.

Inferences

1. **Master-Worker is Good for Request-Based Work:** This approach works well when each unit of work can take different time. Faster workers automatically receive more tasks, while slower workers receive fewer. That adaptive behavior improves utilization compared to rigid static assignment.

2. **MPI_ANY_SOURCE Helps Utilization:** By receiving from any source, the master reacts to whichever worker finishes first instead of waiting in rank order. This removes unnecessary waiting and keeps the task queue moving. It is a practical way to improve throughput in request-driven patterns.

Conclusion

The prime-number master-worker implementation is correct, stable, and easy to follow. It demonstrates a reusable dynamic scheduling model using simple MPI send/receive operations. The experiment confirms that request-based work distribution is practical for distributed integer testing tasks.

QUESTION 5

Question 5: Write a program to find all perfect numbers (numbers equal to sum of proper divisors) up to some maximum value, using `MPI_Recv` for worker requests.

Master loops from 2 to max value and:

a. receives request from any slave (`MPI_ANY_SOURCE`),

b. checks previous result (positive perfect, negative not perfect),

c. sends next number to test.

Slave sends initial request (zero), receives number, checks divisor sum, returns positive if perfect else negative.

Observations

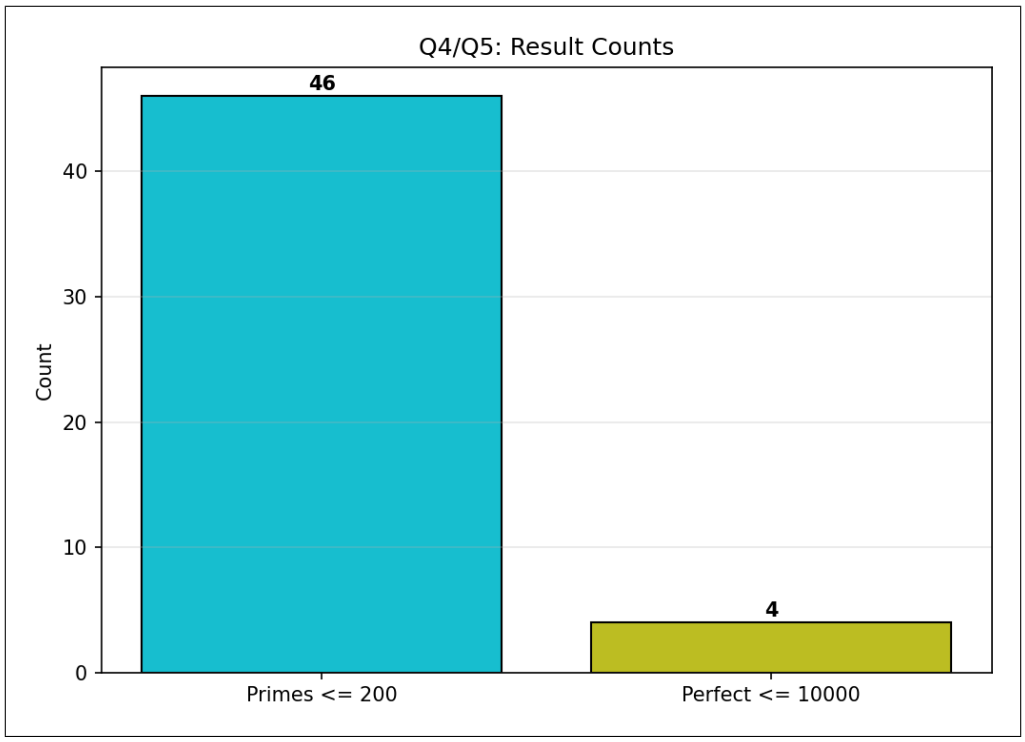
output5.png

1. **Correct Perfect Numbers Found:** For the range up to 10000, the program correctly found 6, 28, 496, and 8128. These are the known perfect numbers in that interval. So the divisor-sum test and result collection are both correct.

2. **Rare Event Behavior:** Perfect numbers are mathematically rare, so only four values appeared in the tested range. This sparse positive output is expected and validates that the

algorithm is not over-reporting candidates. Negative responses dominate the worker reports.

3. **Communication Pattern Reuse:** The same communication framework from Q4 was reused with almost no structural changes. Only the local test kernel changed from primality checking to perfect-number checking. This confirms that the master-worker design is modular and reusable.



Graph 5: Output count comparison for Q4 and Q5

Inferences

1. **Master-Worker Template is Reusable:** The same request/response protocol can solve multiple number-theory tasks without redesigning communication. This reduces development effort and debugging complexity. It is a strong software engineering advantage in parallel programming.

2. **Separation of Concerns is Useful:** Communication handling and mathematical checking are clearly separated in this implementation. That makes the code easier to main-

tain and extend for new checks. Future tasks can be added by replacing only the worker compute function.

Conclusion

The perfect-number MPI solution is correct and demonstrates clean reuse of the dynamic master-worker architecture. It also shows that one communication model can support multiple computational kernels with minimal changes. This pattern is suitable for many distributed search and classification problems.

Submitted to : Dr Saif Nalband

Submitted by : Arpan Goyal

Roll No. : 102303479